



湖南理工学院
Hunan Institute of Science and Technology

实验报告

课 程： 软件架构模式

班 级： 软件 21-3BF

姓 名： 奉恒斌

学 号： 12214801136

日 期： 2024-4-20

桥接模式实验

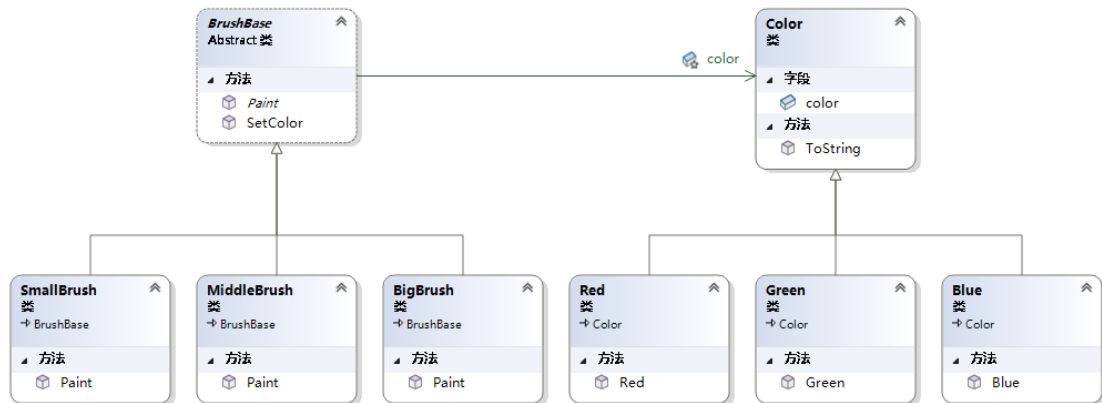
一. 实验目的

- 1) 初步了解和掌握桥接模式（Bridge）的类图结构，以及主要的模式角色；
- 2) 能够利用桥接模式的基本构造，并通过掌握的编程语言，完成实验要求的内容；
- 3) 充分理解桥接模式如何将继承关系转换为组合关系的基本特性。

二. 实验内容

- 1, 以画笔的笔号（大、中、小）和画笔颜色（红、绿、蓝）为基础，实现画笔不同笔号与颜色的功能组合；
- 2, 在实现画笔笔号与颜色的组合中，引入桥接模式，分别定义在该应用场景下的各个模式角色和类图。
- 3, 通过面向对象的程序设计代码，对上述设计进行实现，使得画笔在绘制时可以动态地进行笔号与颜色的组合。

三. 实验过程



3.1 绘图工具类图

如 3.1 的类图所示，一个颜色类被具体的子颜色类实现，由笔刷关联调用得到笔刷的颜色，再绘制出相应大小笔刷的颜色。

```

5 个引用
7  public class Color
8  {
9      public string color;
10     public override string ToString()
11     {
12         return color;
13     }
14 }
2 个引用
15 public class Green : Color
16 {
17     public Green()
18     {
19         this.color = "green";
20     }
21 }
2 个引用
22 public class Red : Color
23 {
24     public Red()
25     {
26         this.color = "red";
27     }
28 }
2 个引用
29 public class Blue : Color
30 {
31     public Blue()
32     {
33         this.color = "red";
34     }
35 }
36

```

图 3.2 颜色类及实现

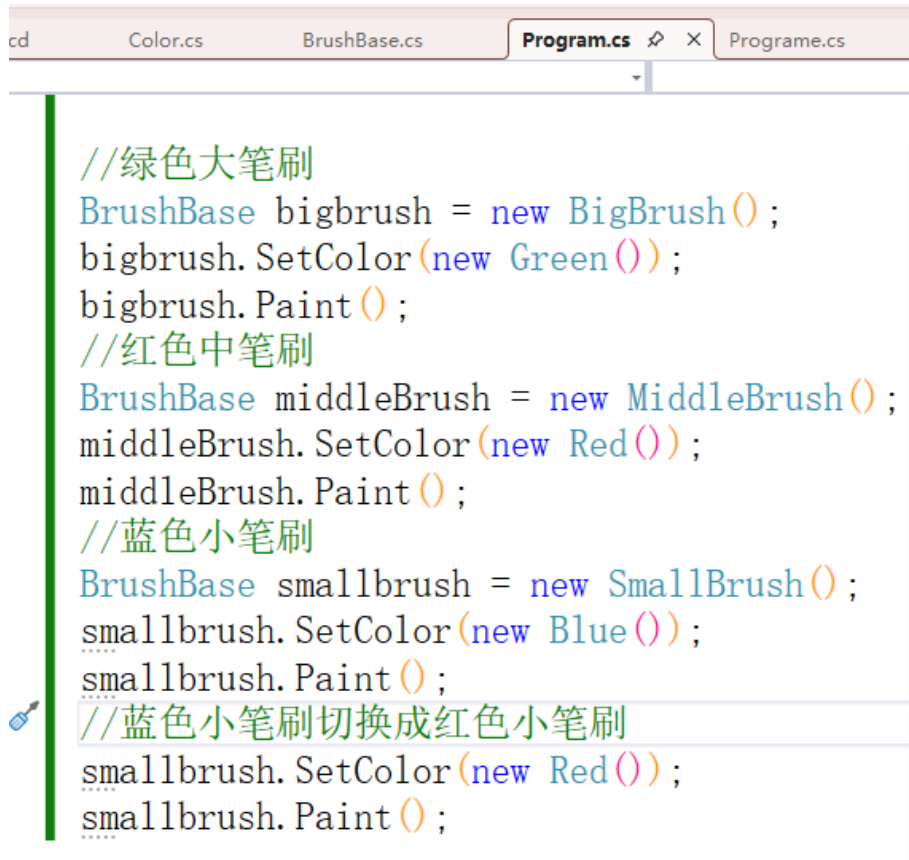
如图 3.2 所示，定义了一个 Color 类，由下面的子类 Green，Red 和 Blue 实现具体的颜色。



图 3.3 笔刷类及实现

如图 3.3 所示，笔刷关联调用颜色，再桥接到具体大小的笔刷上。

四. 调试和运行结果



```
cd      Color.cs      BrushBase.cs      Program.cs  X  Programe.cs
//绿色大笔刷
BrushBase bigbrush = new BigBrush();
bigbrush.SetColor(new Green());
bigbrush.Paint();
//红色中笔刷
BrushBase middleBrush = new MiddleBrush();
middleBrush.SetColor(new Red());
middleBrush.Paint();
//蓝色小笔刷
BrushBase smallbrush = new SmallBrush();
smallbrush.SetColor(new Blue());
smallbrush.Paint();
//蓝色小笔刷切换成红色小笔刷
smallbrush.SetColor(new Red());
smallbrush.Paint();
```

图 4.1 笔刷测试代码



```
Microsoft Visual Studio 调试控制台
BigBrush with green
MiddleBrush with red
SmallBrush with blue
SmallBrush with red
D:\UnityProject\TrainDemo\Brid
要在调试停止时自动关闭控制台。
```

图 4.2 运行结果

如图 4.1、4.2 所示，大笔刷设置成绿色后打印绿色大笔刷，中型笔刷打印了相应的颜色，小笔刷开始设置成蓝色，重新设置后打印出了红色小笔刷。

五. 实验总结

在实验过程中，我首先绘制了画笔的类图，明确了各个模式角色和它们之间的关系。这一过程让我更加清晰地认识到桥接模式是如何将继承关系转换为组合关系的。通过组合关系，我们可以更加灵活地组合不同的功能，从而满足不同的需求。

在实现画笔的笔号和颜色组合时，我引入了桥接模式。通过将笔号与颜色分别抽象为接口，然后创建具体的实现类，我实现了画笔的动态组合。这一过程中，我深刻体会到了桥接模式的优势：它使得代码更加灵活和可扩展，同时降低了类之间的耦合度。